

Adaptive Hybrid RAG System for Question Answering

The goal of this project is to build a complete, research-oriented production-minded Retrieval-Augmented Generation (RAG) system that goes beyond a basic “retrieve documents and ask an LLM” implementation. We want to implement the major components of a modern RAG pipeline ourselves, understand how each component affects the final answer, and run controlled experiments to measure the quality/latency tradeoffs. The final system should support simple factual questions, normal retrieval-based questions, and complex multi-hop questions where answering the query requires retrieving information multiple times. The project should ultimately demonstrate that we understand the complete RAG pipeline — from document ingestion and embeddings all the way through retrieval, reranking, adaptive routing, generation, and evaluation — rather than simply demonstrating familiarity with a framework such as LangChain or LangGraph.

1. Overall System Architecture

The high-level architecture will be:

Documents → Parsing/Cleaning → Chunking → Embeddings → Vector Index + BM25 Index → Hybrid Retrieval → RRF Fusion → Cross-Encoder Reranking → Context Selection → LLM Generation

On top of this retrieval pipeline, we will implement an **Adaptive RAG controller** that examines the query and decides whether it should be answered directly by the LLM, answered using a single retrieval step, or handled using iterative/multi-hop retrieval. For multi-hop questions, the system should be able to decompose the problem into intermediate questions, retrieve evidence for each step, use the retrieved evidence to formulate the next query, and eventually synthesize a final answer.

The important design principle is that the project should expose and evaluate the individual RAG components rather than hiding everything behind a high-level framework. We can use standard libraries for individual components, but the architecture and experimentation should make it clear what is actually happening at every stage.

2. Document Ingestion and Knowledge Base

We will first create a document ingestion pipeline. The system should accept a collection of documents and convert them into a consistent internal representation. The documents should be cleaned, normalized, and divided into meaningful chunks while preserving metadata such as document ID, title, section, source, and chunk position. Chunking should not simply be treated as an arbitrary fixed-size operation; we should experiment with chunk size and overlap and investigate how these choices affect retrieval quality.

Each chunk will receive a unique identifier and metadata so that, when the LLM produces an answer, we can trace the answer back to the specific pieces of evidence that were retrieved. This will also allow us to evaluate retrieval independently from generation.

3. Embedding and Dense Retrieval

For dense retrieval, we will use an embedding model to convert every document chunk into a vector representation. The query will also be converted into the same embedding space, and the system will retrieve candidate chunks based on vector similarity.

The important part here is that we should treat the embedding model as an actual component of the retrieval system rather than just calling an embedding API. We should record which embedding model is being used, embedding dimensions, indexing method, similarity metric, and retrieval latency. We can optionally compare a small number of embedding models to determine whether better semantic representations actually improve retrieval performance on our dataset.

The resulting vectors will be stored in a vector index/database so that we have a persistent, queryable dense-retrieval layer rather than only performing an in-memory demonstration.

4. Sparse Retrieval with BM25

In parallel with dense retrieval, we will maintain a traditional sparse BM25 index over the same document chunks. BM25 is valuable because dense retrieval and lexical retrieval behave differently. Dense retrieval is good at semantic similarity, while BM25 can be particularly effective when the query contains exact terminology, names, numbers, technical phrases, or rare keywords.

For every query, we will therefore produce two candidate sets:

Query → BM25 → candidate documents

and

Query → embedding → vector search → candidate documents

We will preserve the retrieval scores and rankings from both systems so that we can analyze their behavior independently.

5. Hybrid Retrieval and Reciprocal Rank Fusion

The next layer will combine sparse and dense retrieval into a hybrid retrieval system. We will initially use Reciprocal Rank Fusion (RRF) to merge the rankings produced by BM25 and dense retrieval.

The pipeline will therefore be:

BM25 candidates + Dense candidates → RRF → unified candidate set

Rather than assuming that hybrid retrieval is automatically better, we should experimentally compare BM25-only, dense-only, and hybrid retrieval. This gives us a measurable reason for including the hybrid system.

The experiment should report retrieval metrics such as Recall@K, MRR, and nDCG where appropriate. We should also analyze cases where BM25 succeeds but dense retrieval fails, cases where dense retrieval succeeds but BM25 fails, and cases where combining both actually improves the candidate set.

6. Cross-Encoder Reranking

After hybrid retrieval, we will introduce a second-stage reranker. This is one of the most important additions beyond the existing basic RAG implementation.

Instead of immediately sending the top retrieved chunks to the LLM, we will retrieve a larger candidate set — for example, the top 20–50 documents — and pass the query/document pairs through a cross-encoder reranker. The reranker will score how relevant each candidate passage is to the specific query and reorder them.

The pipeline becomes:

BM25 + Dense → RRF → Top 20/50 candidates → Cross-Encoder Reranker → Top 5/10 contexts → LLM

This allows us to study the difference between first-stage retrieval and second-stage relevance ranking. We should measure whether reranking actually improves retrieval metrics and final answer quality, as well as how much additional latency it introduces.

7. Adaptive RAG / Query Routing

Once the underlying retrieval pipeline is working, we will build the Adaptive RAG component. The central idea is that every query does not need the same amount of retrieval.

For a simple question that the LLM can answer reliably, retrieval may be unnecessary. For a straightforward factual question about the knowledge base, one retrieval step may be sufficient. For a complex question requiring multiple pieces of evidence, the system may need several retrieval steps.

The router will therefore classify or otherwise determine the appropriate strategy:

Direct LLM → no retrieval

Single-step RAG → retrieve once → rerank → generate

Multi-hop RAG → retrieve → reason/decompose → retrieve again → synthesize

This component will be based on the Adaptive-RAG paper we are studying/reimplementing, but our implementation will integrate it with our stronger hybrid retrieval and reranking pipeline.

8. Multi-Hop Retrieval and Query Decomposition

For complex queries, we will implement an iterative retrieval process. Instead of retrieving once and hoping the relevant information is present in the context, the system should be able to identify intermediate information that is needed to answer the original question.

For example, a query may implicitly require:

Question → identify entity A → retrieve information about A → identify entity B → retrieve information about B → combine evidence → answer

The system should maintain a state containing the original query, intermediate sub-questions, retrieved documents, evidence, and previous reasoning/retrieval steps.

We should place limits on the number of retrieval iterations so that the system cannot enter an unnecessarily expensive loop. The final answer should be generated only after the system determines that it has sufficient evidence or reaches the maximum allowed number of steps.

9. Context Construction and Grounded Generation

After retrieval and reranking, the system will construct the context provided to the LLM. We should investigate how many chunks should be included, how they should be ordered, and how duplicate or redundant information should be handled.

The generation prompt should explicitly instruct the model to answer using the retrieved evidence and avoid inventing unsupported information. Each answer should retain references to the source chunks used to construct the context so that we can inspect whether the answer is actually grounded in retrieved evidence.

This will allow us to evaluate not only whether the answer is correct, but whether the retrieved evidence actually supports it.

10. Evaluation Framework

A major goal of the project is to avoid having a final result that simply says “the chatbot works.” We want a proper evaluation framework that can compare different RAG configurations.

At the retrieval level, we should measure metrics such as **Recall@K, MRR, and nDCG**. These tell us whether the relevant evidence is actually being retrieved.

At the answer level, we should measure **answer correctness/accuracy, exact match or F1 where appropriate, faithfulness/groundedness, and answer relevance**. We should distinguish retrieval failures from generation failures wherever possible. For example, if the correct evidence was never retrieved, the problem is fundamentally different from a situation where the correct evidence was retrieved but the LLM produced an incorrect answer.

At the systems level, we should measure **end-to-end latency, retrieval latency, reranking latency, number of retrieval calls, input/output tokens, and potentially estimated inference cost**.

This will let us analyze the complete quality-efficiency tradeoff rather than optimizing accuracy in isolation.

11. Ablation and Comparative Experiments

We should run systematic experiments rather than only evaluating the final system. At minimum, we should compare:

LLM-only

BM25 RAG

Dense RAG

Hybrid BM25 + Dense RAG

Hybrid + Reranking

Single-step RAG

Multi-hop RAG

Adaptive RAG

The purpose is to determine exactly which components contribute to performance.

We can additionally perform experiments on chunk size, embedding model, retrieval K, reranking K, and the maximum number of multi-hop retrieval iterations.

For every experiment, we should record retrieval quality, answer quality, latency, and token usage. The final project should therefore contain a clear experimental table showing how each architectural decision affects the system.

For example:

Configuration	Recall@K	MRR	Answer Quality	Faithfulness	Latency	Retrieval Calls
LLM only	—	—				0
BM25						1
Dense						1
Hybrid						1
Hybrid + Reranker						1
Multi-hop						2+
Adaptive RAG						variable

The actual numbers should come from our experiments rather than being predetermined.

12. Vector Database / Indexing Layer

The dense retrieval system should use a proper vector indexing/database layer so that the project demonstrates how embeddings are actually persisted and queried. The vector database should store the embedding, chunk ID, and metadata for each document chunk.

The BM25 index and vector index should be treated as two complementary retrieval systems rather than forcing everything into one database.

The important point is that the vector database is an infrastructure component supporting dense retrieval; it should not become the focus of the project.

13. Serving the System

Once the research implementation is complete, we should expose the final pipeline through a simple API, preferably using FastAPI.

A request such as:

`POST /query`

would contain the user question and potentially configuration parameters. The service would return the final answer, retrieved sources, retrieval strategy selected by the router, number of retrieval iterations, and useful execution metrics.

For example, a response could contain:

```
answer
sources
retrieval_strategy = "multi-hop"
retrieval_calls = 2
latency_ms
token_usage
```

This gives us a clean boundary between the RAG system and a future UI or client.

14. Docker and Reproducibility

The complete application should be containerized so that another person can clone the repository, start the required services, load the knowledge base, and run the RAG API without manually reconstructing the development environment.

The repository should contain a clear README explaining the architecture, setup process, data pipeline, experiments, evaluation methodology, and final findings.

We should prioritize reproducibility over unnecessary infrastructure complexity. The objective is to demonstrate the RAG system, not to build a large Kubernetes deployment.

15. Observability and Experiment Logging

Every query should generate useful execution information. At minimum, we should log the selected RAG strategy, retrieval method, number of retrieved candidates, reranking results, number of retrieval iterations, latency at each stage, and token usage.

This will make it possible to inspect individual failures and understand why the system produced a particular answer.

For example, if Adaptive RAG chooses multi-hop retrieval and takes 2.8 seconds while single-step RAG takes 1.1 seconds, we should be able to determine whether the additional retrieval produced a measurable improvement in answer quality.

16. What We Should NOT Build

We should deliberately avoid turning the project into a collection of GenAI buzzwords. We do not need to add LangChain, LangGraph, multiple vector databases, agents, multi-agent systems, Kubernetes, a large React frontend, or several unrelated LLM frameworks simply because they appear in job descriptions.

The project should focus on understanding and implementing the underlying RAG mechanics. Frameworks can be used internally where they are genuinely useful, but they should not be the selling point of the project.

The core technical story should be:

Embeddings → Sparse/Dense Retrieval → Hybrid Retrieval → RRF → Reranking → Context Construction → Generation → Adaptive Routing → Multi-Hop Retrieval → Evaluation

That is much more valuable than simply saying that we built a “LangChain RAG application.”

17. Division of Work Between Two People

We can divide the project into two major tracks while keeping a shared interface between them.

Person 1 — Retrieval / Knowledge Pipeline: document ingestion, cleaning, chunking, embedding generation, vector indexing, BM25, dense retrieval, hybrid retrieval, RRF, reranking, and retrieval evaluation.

Person 2 — Adaptive RAG / Generation / Evaluation: LLM integration, query routing, direct/single/multi-hop RAG, query decomposition, iterative retrieval, context construction, answer generation, generation evaluation, latency/token measurement, and experiment orchestration.

Both people should understand the complete architecture, and we should define interfaces between components early so that the two tracks can be developed independently and then integrated.

For example, the retrieval module should expose something conceptually like:

```
retrieve(query, top_k) → ranked candidate chunks
```

and the reranking module:

```
rerank(query, candidates, top_k) → ranked contexts
```

while the Adaptive RAG layer should be able to call these components without knowing how BM25, embeddings, or the vector database are internally implemented.

18. Final Deliverable

The final deliverable should be more than a notebook. We should have a clean repository containing the ingestion pipeline, retrieval modules, reranker, Adaptive RAG controller, evaluation framework, API, Docker configuration, experiment results, and documentation.

The repository should include an architecture diagram, setup instructions, example queries, benchmark results, ablation studies, and a discussion of failure cases.

Ideally, the final demonstration should show several queries and explain why the system chose direct generation, single-step RAG, or multi-hop RAG for each one. We should also be able to demonstrate the difference between BM25, dense retrieval, hybrid retrieval, and hybrid-plus-reranking.

The ultimate goal is to have a project that can honestly be described as an **end-to-end adaptive RAG system with hybrid sparse/dense retrieval, vector search, cross-encoder reranking, multi-hop retrieval, and systematic evaluation**, rather than simply a chatbot built around an LLM.

19. Resume-Level Outcome

If we execute the project properly, the resume should emphasize the engineering and ML decisions rather than the frameworks. A strong final description could eventually look like:

Adaptive Hybrid RAG for Multi-Hop Question Answering | Python, PyTorch, Hugging Face, BM25, Dense Retrieval, Vector Search, FastAPI, Docker

- Built an adaptive RAG system routing queries between a) direct LLM inference, b) single-step retrieval, and c) iterative multi-hop retrieval based on query complexity.
- Developed a hybrid retrieval pipeline combining BM25 and dense embeddings with Reciprocal Rank Fusion and cross-encoder reranking for improved evidence retrieval.
- Designed an evaluation framework measuring retrieval recall, ranking quality, answer correctness, faithfulness, latency, token usage, and retrieval efficiency across multiple RAG configurations.
- Conducted controlled ablations of sparse/dense retrieval, reranking, chunking, and adaptive retrieval strategies to quantify quality–efficiency tradeoffs.

The final bullets should only contain results and technologies that we actually implement and measure. In particular, the project will become much stronger once we can replace generic claims such as “improved retrieval” with concrete experimental results.